Міністерство освіти і науки України

Національний університет «Запорізька політехніка»

Кафедра програмних засобів

ЗВІТ

Дисципліна «Проєктування, моделювання та забезпечення якості програмного забезпечення систем»

Робота №1

Тема «Розроблення веб-застосувань з використанням Django»

Виконав варіант 15

Студент КНТ-116м Онищенко О.А.

Прийняли

Викладач Табунщик Г.В.

2026

МЕТА

Ознайомитися із основними можливостям фреймворку Django, навчитися створювати та запускати проєкти із використанням утиліти Supervisor та Nginx сервера.

ЗАВДАННЯ

Порядок виконання роботи:

2.1. Застосувавши конспект лекцій та додаткову літературу, вивчити принципи створення проєктів у Django.

2.2. Згідно з наведеним прикладом створити вебзастосунок за індивідуальним завданням викладача.

Варіант 1. Створити базу даних лікарів та форми додавання і звільнення персоналу.

Варіант 2. Створити базу даних пацієнтів. Створити форми надходження та виписки пацієнтів.

Варіант 3. Створити базу даних медикаментів. Створити форми за призначенням та відділенням.

Варіант 4. Створити базу даних медичних інструментів. Створити форми за видом та призначенням інструменту.

Варіант 5. Створити графік чергувань лікарів. Створити форми відділення для чергування та замін.

Варіант 6. Створити базу даних карток пацієнтів. Створити форми за лікарем та хворобою.

Варіант 7. Створити базу даних медичної техніки. Створити форми за призначенням та відділенням.

Варіант 8. Створити базу даних медикаментів. Створити форми закупівлі та використання медикаментів.

Варіант 9. Створити базу даних пацієнтів. Створити форми пошуку пацієнтів за лікарем та хворобою.

Варіант 10. Створити графік проведення операцій. Створити форми за лікарем та діагнозом.

2.3. Оформити звіт. Звіт з лабораторної роботи повинен містити:

- мету роботи;
- завдання на виконання роботи;
- тексти програм;
- результати роботи;
- висновки.

2.4. Відповісти на контрольні запитання.

Темою всієї роботи є «настільний анти-стрес клікер».

Завданням на поточну роботу є: Веб інтерфейс для моніторингу статусу стресу всіх клієнтів.

КОД

Жирним шрифтом інколи виділено місця, які було змінено.

templates/users/base.html

```
{% load static %}
<!DOCTYPE html>
<html lang="uk">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-
scale=1.0">
    <title>{% block title %}Anti-Stress Clicker{% endblock %}</title>
    <link rel="stylesheet" href="{% static 'css/style.css' %}">
</head>
<body>
    <header class="site-header">
        <div class="container header-content">
            <a class="logo" href="{% url 'home' %}">Anti-Stress
Clicker</a>
            <nav class="main-nav">
```

```
                <a href="{% url 'home' %}">Головна</a>
                <a href="{% url 'customers' %}">Моніторинг клієнтів</a>
                <a href="{% url 'profile' %}">Особистий кабінет</a>
            </nav>
        </div>
    </header>

    <main class="container">
        {% block content %}{% endblock %}
    </main>

    <footer class="site-footer">
        <div class="container">
            <p>Anti-Stress Clicker — навчальний проєкт</p>
        </div>
    </footer>
</body>
</html>
```

templates/users/customer_list.html

```
{% extends 'users/base.html' %}

{% block title %}Моніторинг клієнтів — Anti-Stress Clicker{% endblock
%}

{% block content %}
<section class="page-header">
    <h1>Моніторинг клієнтів</h1>
    <p>Дані про всіх користувачів в одному місці.</p>
</section>

<section class="card">
    <h2>Дані користувачів</h2>
    <div class="table-wrapper">
        <table class="activity-table">
            <thead>
                <tr>
                    <th>Ім’я</th>
                    <th>Рівень стресу</th>
                    <th>Загальна кількість натискань</th>
                    <th>Середня швидкість натискань</th>
                    <th>Останній час активності</th>
                </tr>
            </thead>
            <tbody>
                {% for user in users_data %}
                <tr>
                    <td>{{ user.full_name }}</td>
                    <td>
                        {% if user.stress_level == "LOW" %}
                            Низький
                        {% elif user.stress_level == "MEDIUM" %}
                            Середній
                        {% elif user.stress_level == "HIGH" %}
```

```
                            Високий
                        {% endif %}
                    </td>
                    <td>{{ user.total_clicks }}</td>
                    <td>{{ user.average_clicking_speed }}</td>
                    <td>{{ user.last_activity }}</td>
                </tr>
                {% endfor %}
            </tbody>
        </table>
    </div>
</section>
{% endblock %}
```

models.py

```
from dataclasses import dataclass
from enum import StrEnum
from datetime import datetime

from django.db import models
from django.core.validators import RegexValidator

"""
class Customer(models.Model):
    class StressLevel(models.TextChoices):
        LOW = "LOW", "Низький"
        MEDIUM = "MEDIUM", "Середній"
        HIGH = "HIGH", "Високий"

    full_name = models.CharField(
        max_length=255,
        validators=[
            RegexValidator(
                regex=r".*\s+.*",
                message="Значення імені має містити хочаб один
пробіл.",
            )
        ],
    )
    stress_level = models.CharField(
        max_length=12, choices=StressLevel.choices,
default=StressLevel.LOW
    )
    total_clicks = models.PositiveIntegerField(default=0)
    average_clicking_speed = models.FloatField(default=0.0)
    last_activity = models.DateTimeField(auto_now=True)
"""

class StressLevel(StrEnum):
    LOW = "LOW"
    MEDIUM = "MEDIUM"
    HIGH = "HIGH"

@dataclass
```

```
class Customer:
    full_name: str
    stress_level: StressLevel = StressLevel.LOW
    total_clicks: int = 0
    average_clicking_speed: float = 0.0
    last_activity: datetime | None = None
```

urls.py

```
from django.urls import path

from . import views

urlpatterns = [
    path("", views.home, name="home"),
    path("profile/", views.profile, name="profile"),
    path("customers/", views.customer_details, name="customers"),
]
```

views.py

```
from django.http import HttpRequest
from django.shortcuts import render

from .models import Customer, StressLevel
from datetime import datetime


def home(request):
    return render(request, "users/home.html")


def customer_details(request: HttpRequest):
    users_data: list[Customer] = [
        Customer(
            full_name="Bertha King",
            stress_level=StressLevel.LOW,
            total_clicks=12,
            average_clicking_speed=3.2,
            last_activity=datetime.now(),
        ),
        Customer(
            full_name="Anthony Ward",
            stress_level=StressLevel.MEDIUM,
            total_clicks=94,
            average_clicking_speed=6.5,
            last_activity=datetime.now(),
        ),
        Customer(
            full_name="Frederick Owens",
            stress_level=StressLevel.HIGH,
            total_clicks=98,
```

```
            average_clicking_speed=2.2,
            last_activity=datetime.now(),
        ),
        Customer(
            full_name="Lora Saunders",
            stress_level=StressLevel.LOW,
            total_clicks=44,
            average_clicking_speed=8.3,
            last_activity=datetime.now(),
        ),
        Customer(
            full_name="Tillie Graves",
            stress_level=StressLevel.MEDIUM,
            total_clicks=59,
            average_clicking_speed=7.0,
            last_activity=datetime.now(),
        ),
        Customer(
            full_name="Alejandro Williams",
            stress_level=StressLevel.HIGH,
            total_clicks=91,
            average_clicking_speed=5.3,
            last_activity=datetime.now(),
        ),
        Customer(
            full_name="Beulah Owens",
            stress_level=StressLevel.LOW,
            total_clicks=33,
            average_clicking_speed=3.5,
            last_activity=datetime.now(),
        ),
    ]
    context: dict = {
        "users_data": users_data,
    }
    return render(request, "users/customer_list.html", context)


def profile(request):
    user_info = {
        "full_name": "Олена Коваленко",
        "username": "olena.kovalenko",
        "date_joined": "12.09.2025",
    }

    statistics = {
        "total_clicks": 1840,
        "completed_sessions": 24,
        "average_clicks_per_session": 77,
        "last_activity": "18.03.2026",
    }

    activity_history = [
        {"date": "18.03.2026", "clicks": 92, "duration": "12 хв"},
        {"date": "15.03.2026", "clicks": 65, "duration": "8 хв"},
        {"date": "12.03.2026", "clicks": 110, "duration": "15 хв"},
        {"date": "09.03.2026", "clicks": 48, "duration": "6 хв"},
        {"date": "05.03.2026", "clicks": 81, "duration": "10 хв"},
```

```
    ]

    context = {
        "user_info": user_info,
        "statistics": statistics,
        "activity_history": activity_history,
    }
    return render(request, "users/profile.html", context)
```

РЕЗУЛЬТАТИ

Результатом є сторінка у веб-застосунку, яка показує дані користувачів у таблиці. Деталі на рисунку 1.1.

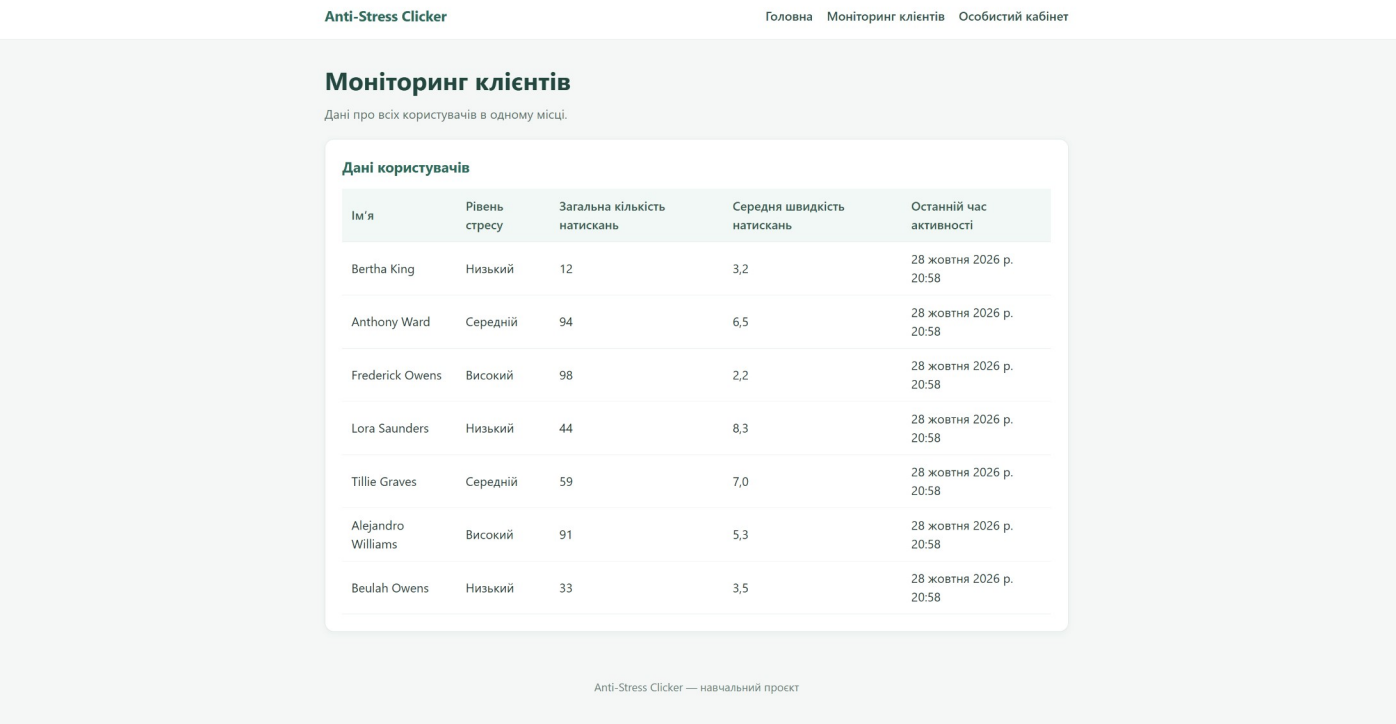


Рисунок 1.1 — Додана сторінка

ВИСНОВКИ

У результаті роботи, завдяки Господу Ісусу Христу, було створено сторінку з випадково згенерованими даними, які відображені таблицею.

Як результат було отримано базове розуміння роботи застосунків на фреймворку Django.

ПИТАННЯ

1 Перелічіть можливості Django.

Можливості Django наступні:

- створення вебсайтів;
- робота з базами даних;
- автентифікація користувачів;
- робота з формами;
- маршрутизація URL;
- адміністративна панель.

2 Які головні принципи реалізовує Django?

Django реалізовує наступні принципи:

- DRY (Don’t Repeat Yourself): уникнення дублювання коду;
- швидке прототипування;
- чіткий поділ відповідальностей.

3 Для чого використовується віртуальне середовище? Як його створити?

Віртуальне середовище потрібне для ізоляції залежностей проєкту. Для створення та активації віртуального середовища можна використати наступні команди:

```
python -m venv .venv
.venv\Scripts\activate
```

4 Дайте визначення WSGI.

WSGI (Web Server Gateway Interface) – стандарт взаємодії між веб-застосункком на Python та веб сервером.

5 Які проблеми вірішує Supervisor?

Supervisor – це програма для керування фоновими процесами: вона запускає їх, перезапускає після збоїв та контролює їхню роботу.

6 Наведіть загальний опис архітектури Django.

Архітектура Django використовує модель MVT (Model-View-Template). Model відповідає за дані, View за обробку запитів, Template за відображення сторінок.

7 Як створити застосування та проект у Django? Чим вони відрізняються?

Проєкт містить загальні налаштування сайту та об’єднує застосунки.

Застосунок відповідає за окремий функціонал.

Створення:

```
django-admin startproject Project_Name
python manage.py startapp App_Name
```

8 Які можливості існують у Django для роботи з БД?

Можливості, що надає Django для роботи з базами даних:

- ORM (Object Relational Mapping) для роботи з моделями без необхідності ручного написання SQL запитів
- створення міграцій
- виконання запитів
- фільтрація
- додавання, зміна та видалення записів

Підтримує SQLite, PostgreSQL, MySQL, Oracle.

9 Як створюються URL для сторінок веб-застосування у Django?

Маршрути визначаються у urls.py за допомогою методу path() або re_path(). Вони пов’язуються з відповідними представленнями (views):

```
path("customers/", views.customer_list, name="customer_list")
```

10 Що таке форма у Django та HTML?

HTML форма збирає дані від користувача та надсилає їх серверу.

Django форма допомагає створювати поля, перевіряти дані та обробляти результати надсилання.